BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT HƯNG YÊN



ĐỒ ÁN 3

DRIVEFOCUS
GIÁM SÁT TÀI XẾ BUỒN NGỦ VÀ MẤT TẬP TRUNG

NGÀNH: KHOA HỌC MÁY TÍNH
CHUYÊN NGÀNH: TRÍ TUỆ NHÂN TẠO VÀ KHOA HỌC DỮ LIỆU

SINH VIÊN: **TRẦN GIA HUY**
MÃ SINH VIÊN: **10123169**
MÃ LỚP: **12423TN**
GIẢNG VIÊN HƯỚNG DẪN: PGS.TS. NGUYỄN MINH TIẾN

HƯNG YÊN – 2026

Nhận xét của giảng viên hướng dẫn:

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Ký và ghi họ tên

LỜI CAM ĐOAN

Em xin cam đoan đồ án 3 là kết quả thực hiện của em dưới sự hướng dẫn của PGS.TS. Nguyễn Minh Tiến.

Những phần sử dụng tài liệu tham khảo trong đồ án đã được nêu rõ trong phần tài liệu tham khảo. Các kết quả trình bày trong đồ án và chương trình xây dựng được hoàn toàn là kết quả do bản thân em thực hiện.

Nếu vi phạm lời cam đoan này, em xin chịu hoàn toàn trách nhiệm trước khoa và nhà trường.

Hưng Yên, ngày 9 tháng 11 năm 2026

Sinh viên

Trần Gia Huy

LỜI CẢM ƠN

Để hoàn thành đồ án này, trước hết em xin gửi lời cảm ơn tới bộ môn Khoa học máy tính, Khoa Công nghệ thông tin, Trường Đại học Công nghệ Kỹ thuật Hưng Yên đã tạo điều kiện thuận lợi cho em thực hiện đồ án môn học này.

Đặc biệt, em xin chân thành cảm ơn thầy Nguyễn Minh Tiến đã tận tình hướng dẫn và góp ý cho em trong suốt thời gian thực hiện đồ án.

Do trình độ còn hạn chế, đồ án khó tránh khỏi thiếu sót. Em rất mong nhận được ý kiến nhận xét và góp ý của thầy cô về những kết quả trình bày trong bài.

Em xin trân trọng cảm ơn!

MỤC LỤC

CHƯƠNG 1: TỔNG QUAN BÀI TOÁN 7
1.1 Bài toán 7
1.2 Trình bày dữ liệu bài toán 8
1.3 Tiền xử lý dữ liệu 9
1.4 Trực quan dữ liệu 10
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT 11
2.1 Thị giác máy tính và giám sát tài xế 11
2.1.1 Giới thiệu về thị giác máy tính 11
2.1.2 Các dấu hiệu buồn ngủ và mất tập trung 11
2.1.3 Trí tuệ nhân tạo trên thiết bị biên và chip K230 11
2.2 OpenCV và MediaPipe 12
2.2.1 Giới thiệu về OpenCV 12
2.2.2 MediaPipe 12
2.2.3 Cài đặt 12
2.3 Các đại lượng hình học trên khuôn mặt 13
2.3.1 Tỉ số mở mắt EAR 13
2.3.2 PERCLOS 13
2.3.3 Tỉ số mở miệng MAR 13
2.3.4 Tư thế đầu 14
2.4 Luật bàn tay gần đầu 14
2.5 Ngân sách chú ý và máy trạng thái 14
2.6 Hạ tầng dữ liệu của Traffic-SM 15
2.7 Chỉ số đánh giá 15
CHƯƠNG 3: NỘI DUNG THỰC HIỆN VÀ KẾT QUẢ 16
3.1 Kiến trúc hệ thống 16

3.2 Mã nguồn các bộ đo 17
3.2.1 Tỉ số mở mắt 17
3.2.2 Thời gian nhắm mắt liên tục 17
3.2.3 Xác định xe đang chạy 18
3.2.4 Ngân sách chú ý 19
3.3 Thực nghiệm và kết quả 20
3.3.1 Kiểm thử tự động 20
3.3.2 Thời gian nhắm mắt và mức cảnh báo 20
3.3.3 Ngân sách chú ý 22
3.3.4 Đánh giá luật bàn tay gần đầu trên State Farm 23
3.3.5 Hiệu năng 25
3.4 Phần máy chủ và kết nối với Traffic-SM 26
3.5 Sản phẩm 28
KẾT LUẬN 30
TÀI LIỆU THAM KHẢO 32

DANH MỤC HÌNH ẢNH

Hình 1.1: Mười lớp hành vi c0 đến c9 của cùng một tài xế trong bộ State Farm 10
Hình 1.2: Tỉ lệ ảnh phát hiện được theo từng lớp trên tập test của State Farm 10
Hình 2.1: Sáu điểm quanh mắt và công thức EAR 13
Hình 3.1: Kiến trúc hệ thống DriveFocus 16
Hình 3.2: Máy trạng thái hiển thị mức cảnh báo 17
Hình 3.3: Mã nguồn tính EAR từ sáu điểm theo toạ độ điểm ảnh 17
Hình 3.4: Mã nguồn cập nhật thời gian nhắm mắt liên tục 18
Hình 3.5: Mã nguồn xác định trạng thái chuyển động của xe 19
Hình 3.6: Mã nguồn ngân sách chú ý 20
Hình 3.7: Mức đạt được và độ trễ cảnh báo theo thời gian nhắm mắt 21
Hình 3.8: EAR, thời gian nhắm liên tục và mức cảnh báo trên video thử 22
Hình 3.9: Ngân sách chú ý khi nhìn lệch liên tục và khi nhắn tin xen kẽ liếc đường 23
Hình 3.10: Precision, recall và F1 theo ngưỡng k trên tập val 24
Hình 3.11: Tỉ lệ ảnh bị đánh dấu theo từng lớp trên tập test 24
Hình 3.12: Một số kết quả: tai (vuông xanh), miệng (tam giác hồng), bàn tay (tròn vàng) và kết luận của luật 25
Hình 3.13: Thời gian xử lý mỗi khung trên CPU 26
Hình 3.14: Mã nguồn chuyển một điểm GPS của thiết bị sang bản ghi của Traffic-SM 27
Hình 3.15: Giao diện demo ở chế độ riêng tư tại mức 0, mức 2 và mức 4 28

CHƯƠNG 1: TỔNG QUAN BÀI TOÁN

1.1 Bài toán

Buồn ngủ và mất tập trung khi lái xe là hai nguyên nhân thường gặp trong các vụ tai nạn giao thông. Nếu đặt một camera trong xe hướng về phía tài xế thì có thể dùng thị giác máy tính để nhận ra sớm các dấu hiệu này và nhắc tài xế ngay lúc đó. Đề tài DriveFocus xây dựng một hệ thống như vậy, gồm hai phần. Phần đặt trên xe (DF-Edge) xử lý ảnh và tự đưa ra cảnh báo, không cần kết nối mạng. Phần máy chủ (DF-Cloud) nhận các sự kiện đã ẩn danh và chuyển dữ liệu GPS sang hệ thống phân tích giao thông Traffic-SM mà nhóm đã làm trước đó.

Đầu vào của hệ thống là ảnh từ camera trong cabin và tốc độ xe lấy từ GPS. Đầu ra gồm:

- Mức cảnh báo từ 0 đến 4, cập nhật theo từng khung hình. Mức 4 là báo động, dùng cho trường hợp nguy hiểm nhất như nhắm mắt liên tục từ 3 giây trở lên.
- Các sự kiện: loại sự kiện (nhắm mắt, buồn ngủ, nhìn lệch, dùng điện thoại...), mức cao nhất, thời lượng và quãng đường xe đã đi trong lúc xảy ra sự kiện.
- Hệ thống chỉ cảnh báo khi xe đang chạy. Khi xe dừng đèn đỏ thì không cảnh báo.

Trong đề tài này, em không dùng mạng nơ-ron để phân loại trực tiếp hành vi tài xế. Các mô hình có sẵn của MediaPipe chỉ được dùng để tìm khuôn mặt, các điểm mốc trên mặt, bàn tay và tư thế cơ thể. Từ các điểm này em tính ra độ mở mắt, độ mở miệng, góc quay đầu và vị trí tay so với đầu, sau đó dùng các quy tắc có ngưỡng và thời lượng để quyết định mức cảnh báo. Em chọn cách này vì bộ dữ liệu State Farm không có nhãn nhắm mắt hay ngáp, vì các quy tắc dễ giải thích và dễ chỉnh ngưỡng, và vì phần quyết định chạy rất nhanh (khoảng 0,13 ms cho mỗi khung hình), phù hợp để đưa lên thiết bị nhúng như chip Kendryte K230.

Một số khó khăn của bài toán:

- Hệ thống phải hạn chế báo nhầm. Chớp mắt, liếc gương, đeo kính râm hay dừng đèn đỏ đều không phải tình huống nguy hiểm.

- Khi không nhìn rõ mắt (kính râm, thiếu sáng, quay mặt đi) thì không được coi là tài xế đang nhắm mắt.
- Dữ liệu công khai khác với điều kiện thực tế: ảnh State Farm được chụp từ phía ghế phụ nên tài xế bị nhìn nghiêng, trong khi các thuật toán dựa trên mắt cần ảnh mặt nhìn thẳng.
- Thiết bị đặt trên xe có bộ nhớ và năng lực tính toán hạn chế.

1.2 Trình bày dữ liệu bài toán

Đề tài sử dụng một số nguồn dữ liệu khác nhau, mỗi nguồn dùng cho một phần của hệ thống (Bảng 1.1).

Bảng 1.1: Các nguồn dữ liệu và vai trò

Nguồn	Nội dung	Quy mô	Vai trò trong đề tài
State Farm Distracted Driver	Ảnh 640×480, 10 lớp hành vi, camera đặt phía ghế phụ, ban ngày	22.424 ảnh, 26 tài xế	Đánh giá luật bàn tay gần đầu; đo tỉ lệ phát hiện của các mô hình
Ảnh chân dung mẫu của MediaPipe	Một ảnh mặt nhìn thẳng, mắt mở	1 ảnh	Kiểm thử phần cảm biến; giả lập mắt nhắm bằng cách phủ màu da lên vùng mắt
fake_gps.csv của Traffic-SM	GPS mô phỏng trên đường thật ở TP.HCM, gồm ô tô và xe máy	36.941 dòng, 30 tài xế	Cung cấp tốc độ và lộ trình khi chạy thử với Traffic-SM
NTHU-DDD, YawDD, DMD	Video buồn ngủ, ngáp, mất tập trung	Chưa tải	Dự kiến dùng để chỉnh ngưỡng; phải đăng ký mới tải được

Bộ State Farm Distracted Driver Detection là dữ liệu của một cuộc thi trên Kaggle năm 2016, gồm ảnh của 26 tài xế thực hiện mười hành vi từ c0 đến c9. Em dùng lại cách chia dữ liệu theo tài xế của bài tập lớn trước (cùng tệp manifest) để mỗi tài xế chỉ thuộc một tập: 18 tài xế cho tập huấn luyện, 4 tài xế cho tập kiểm định (val, 3.915 ảnh) và 4 tài xế cho tập kiểm tra (test, 3.052 ảnh). Do đề tài không huấn luyện mô hình nên tập huấn luyện không được dùng. Tập val dùng để chọn ngưỡng, tập test dùng để báo cáo kết quả.

Mười lớp hành vi của bộ dữ liệu:

- c0: lái xe an toàn.
- c1, c3: nhắn tin bằng tay phải, bằng tay trái; c2, c4: gọi điện bằng tay phải, bằng tay trái.
- c5: chỉnh radio; c6: uống nước.
- c7: với ra phía sau; c8: chải tóc hoặc trang điểm; c9: nói chuyện với người ngồi bên cạnh.

Dữ liệu GPS lấy từ tệp fake_gps.csv do bộ mô phỏng của Traffic-SM sinh ra. Mỗi dòng gồm mã tài xế, thời điểm, vĩ độ, kinh độ, hướng đi, tốc độ (m/s), độ chính xác và loại phương tiện. Em lấy lộ trình của một tài xế ô tô làm nguồn tốc độ cho thiết bị giả lập, sau đó gửi ngược dữ liệu này vào cổng nhận dữ liệu của Traffic-SM để kiểm tra luồng xử lý.

1.3 Tiền xử lý dữ liệu

- Ảnh được đọc bằng OpenCV ở dạng BGR và chuyển sang RGB trước khi đưa vào MediaPipe. Các khoảng cách đều tính theo toạ độ điểm ảnh, vì toạ độ chuẩn hoá (0 đến 1) làm sai tỉ lệ khi chiều rộng và chiều cao ảnh khác nhau.
- Các bộ đếm thời gian dùng thời điểm thực của khung hình thay vì đếm số khung, nên kết quả không bị ảnh hưởng khi tốc độ khung hình thay đổi.
- Mỗi số đo có ba trạng thái: mở, nhắm hoặc không xác định (khi mất khuôn mặt hoặc vùng mắt quá tối). Trạng thái không xác định không làm tăng hay giảm bộ đếm và không được tính vào PERCLOS.
- Hiệu chỉnh theo từng người: độ mở mắt bình thường lấy bằng trung vị EAR trong khoảng đầu chuyến đi; tư thế đầu khi nhìn thẳng lấy bằng trung vị các góc khi xe chạy từ 30 km/h trở lên.
- Dữ liệu GPS được đổi sang định dạng của Traffic-SM: trường time theo múi giờ UTC+7, trường t_timestamp có dạng phút:giây và mã tài xế được thay bằng mã chuyến đi.
- Với State Farm, em dùng ảnh gốc 640×480 của tập val và tập test, không thu nhỏ, để các mô hình có đủ độ phân giải.

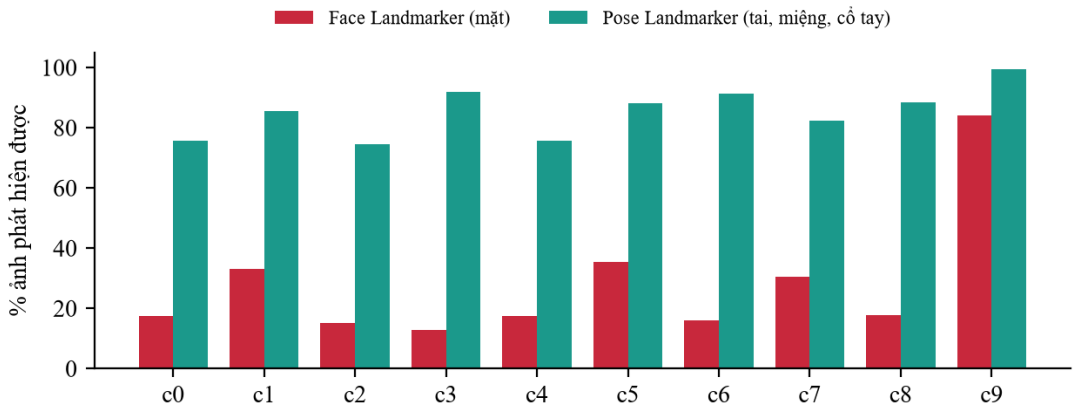
1.4 Trực quan dữ liệu

Hình 1.1 là mười lớp hành vi của cùng một tài xế. Do camera đặt phía ghế phụ nên tài xế luôn bị nhìn nghiêng, chỉ thấy được một bên mặt và một bên tai, bàn tay phía xa thường bị che khuất.



Hình 1.1: Mười lớp hành vi c0 đến c9 của cùng một tài xế trong bộ State Farm

Em chạy hai mô hình của MediaPipe trên 3.052 ảnh của tập test và thống kê tỉ lệ phát hiện theo từng lớp (Hình 1.2). Face Landmarker chỉ tìm thấy khuôn mặt trong 842 ảnh (27,6%). Lớp c9 có tỉ lệ cao nhất (84,2%) vì tài xế quay mặt về phía camera khi nói chuyện, còn lớp c0 chỉ đạt 17,6%. Trong các ảnh tìm thấy mặt, chỉ có 17 ảnh nhìn rõ mắt để tính EAR. Pose Landmarker (mô hình tư thế cơ thể) cho kết quả tốt hơn, tìm được tư thế trong 85,0% số ảnh.



Hình 1.2: Tỉ lệ ảnh phát hiện được theo từng lớp trên tập test của State Farm

Như vậy không thể đánh giá các thuật toán dựa trên mắt (EAR, PERCLOS) bằng bộ dữ liệu này. Em chỉ dùng State Farm để đánh giá luật bàn tay gần đầu, với mô hình tư thế cơ thể thay cho mô hình khuôn mặt.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 Thị giác máy tính và giám sát tài xế

2.1.1 Giới thiệu về thị giác máy tính

Thị giác máy tính là lĩnh vực nghiên cứu cách để máy tính thu nhận, xử lý và hiểu nội dung của ảnh và video. Các bài toán thường gặp gồm phân loại ảnh, phát hiện đối tượng, ước lượng tư thế và theo dõi chuyển động. Trong bài toán giám sát tài xế, các kỹ thuật này được dùng để biết tài xế đang nhìn về đâu, mắt đang mở hay nhắm và tay đang làm gì.

2.1.2 Các dấu hiệu buồn ngủ và mất tập trung

- Nhắm mắt lâu và tỉ lệ thời gian nhắm mắt cao (PERCLOS) là các dấu hiệu buồn ngủ hay được sử dụng.
- Ngáp và gục đầu là dấu hiệu phụ, thường xuất hiện khi mệt.
- Đầu hoặc mắt hướng ra khỏi mặt đường trong thời gian dài là dấu hiệu mất tập trung.
- Tay đưa lên gần tai hoặc miệng có thể là gọi điện, uống nước, ăn uống hoặc hút thuốc; cúi đầu nhìn xuống lâu có thể là nhắn tin.

Các dấu hiệu trên chỉ có ý nghĩa khi kéo dài đủ lâu. Một lần chớp mắt thường ngắn hơn nửa giây, một lần liếc gương thường ngắn hơn một giây. Vì vậy các quy tắc trong đề tài đều có thêm điều kiện về thời lượng.

2.1.3 Trí tuệ nhân tạo trên thiết bị biên và chip K230

Khi xử lý ngay trên thiết bị đặt trong xe, cảnh báo không phải chờ đường truyền, video quay khuôn mặt tài xế không phải gửi đi nơi khác và hệ thống vẫn chạy khi mất sóng. Chip Kendryte K230 có hai lõi RISC-V (một lõi 1,6 GHz có tập lệnh vector chạy hệ điều hành thời gian thực, một lõi 800 MHz chạy Linux), bộ tăng tốc suy luận KPU hỗ trợ INT8 và INT16 (theo nhà sản xuất là khoảng 6 TOPS), ba cổng camera MIPI CSI và bộ xử lý ảnh hỗ trợ cảm biến RGB-IR. Bộ SDK của K230 có sẵn các ví dụ nhận diện khuôn mặt, điểm mốc, tư thế đầu, hướng nhìn và phát hiện bàn tay, có

thể dùng thay MediaPipe khi chuyển từ máy tính sang chip. Trong phạm vi đồ án này em chưa chạy được trên chip K230, các số đo trong báo cáo đều đo trên CPU máy tính.

2.2 OpenCV và MediaPipe

2.2.1 Giới thiệu về OpenCV

OpenCV là thư viện mã nguồn mở cho thị giác máy tính, hỗ trợ đọc ghi ảnh và video, xử lý ảnh, vẽ lên ảnh và giải các bài toán hình học như PnP. Trong đề tài, OpenCV được dùng để đọc camera và video, vẽ thông tin lên khung hình và tính tư thế đầu.

2.2.2 MediaPipe

MediaPipe là bộ công cụ mã nguồn mở của Google dành cho các ứng dụng thị giác chạy thời gian thực. Đề tài sử dụng ba mô hình thông qua Tasks API:

- Face Landmarker: tìm khuôn mặt và 478 điểm mốc (có cả mống mắt), đồng thời trả về ma trận biến đổi 4×4 mô tả tư thế đầu.
- Hand Landmarker: trả về 21 điểm mốc cho mỗi bàn tay.
- Pose Landmarker: trả về 33 điểm mốc trên cơ thể như mũi, tai, miệng, vai và cổ tay. Mô hình này được dùng cho ảnh chụp nghiêng khi Face Landmarker không tìm thấy mặt.

Các mô hình được dùng nguyên bản, không huấn luyện lại. Phiên bản MediaPipe trong đề tài là 1.2.2.

2.2.3 Cài đặt

- Cài thư viện bằng pip: pip install mediapipe opencv-python numpy pyyaml.
- Tải các tệp mô hình (.task) bằng script scripts/download_models.py.
- Phiên bản Python sử dụng là 3.11.

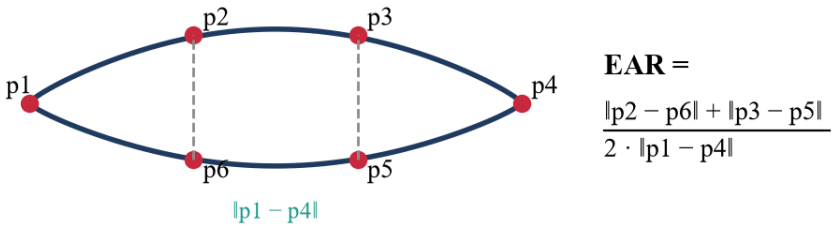
2.3 Các đại lượng hình học trên khuôn mặt

2.3.1 Tỉ số mở mắt EAR

Tỉ số mở mắt EAR (Eye Aspect Ratio) do Soukupová và Čech đề xuất, được tính từ sáu điểm quanh mắt (Hình 2.1):

$$EAR = (\|p2 - p6\| + \|p3 - p5\|) / (2 \cdot \|p1 - p4\|)$$

Trong đó p1 và p4 là hai khoé mắt, hai cặp (p2, p6) và (p3, p5) nằm trên mi trên và mi dưới. Khi mắt mở EAR thường vào khoảng 0,2 đến 0,35, khi mắt nhắm EAR gần bằng 0. Vì độ mở mắt khác nhau giữa từng người và phụ thuộc vào góc camera, em chia EAR cho EAR lúc mắt mở của chính tài xế đó và coi là mắt nhắm khi giá trị này nhỏ hơn 0,5.



Hình 2.1: Sáu điểm quanh mắt và công thức EAR

2.3.2 PERCLOS

PERCLOS là tỉ lệ thời gian mắt nhắm trong một khoảng thời gian, thường là một phút. Em tính PERCLOS theo thời gian thực (cộng độ dài các khoảng mắt nhắm thay vì đếm số khung), bỏ các khoảng không xác định ra khỏi mẫu số và chỉ sử dụng kết quả khi cửa sổ đã đủ dài và phần quan sát được chiếm ít nhất 80%. Cách tính này em tham khảo từ dự án mã nguồn mở driver-state-detection.

2.3.3 Tỉ số mở miệng MAR

MAR (Mouth Aspect Ratio) bằng khoảng cách giữa môi trên và môi dưới chia cho chiều rộng miệng. Một lần ngáp được tính khi MAR lớn hơn ngưỡng liên tục từ 1,5 giây trở lên. Số lần ngáp trong 10 phút được dùng làm dấu hiệu mệt mỏi.

2.3.4 Tư thế đầu

Tư thế đầu được mô tả bằng ba góc: yaw (quay trái phải), pitch (cúi, ngửa) và roll (nghiêng). Em lấy ba góc này từ ma trận biến đổi 4×4 mà Face Landmarker tính được khi khớp toàn bộ 468 điểm với mô hình mặt chuẩn. Ngoài ra em cũng thử giải bài toán PnP (Perspective-n-Point) từ sáu điểm. Trên ảnh mẫu nhìn thẳng, ma trận của MediaPipe cho góc pitch khoảng 4 độ, còn cách PnP sáu điểm cho khoảng 18 độ do bị ảnh hưởng bởi hình dạng khuôn mặt của từng người. Sai lệch cố định có thể loại bỏ khi hiệu chỉnh tư thế nhìn thẳng, nhưng độ nhiễu thì không, nên em chọn dùng ma trận của MediaPipe. Trong đề tài, pitch dương nghĩa là cúi xuống.

2.4 Luật bàn tay gần đầu

Khi gọi điện, uống nước, ăn hoặc hút thuốc, tài xế đều đưa tay lên gần đầu. Luật này đo khoảng cách từ tâm bàn tay đến tai và đến miệng, rồi chia cho kích thước đầu để kết quả không phụ thuộc vào khoảng cách từ người tới camera. Nếu khoảng cách nhỏ hơn ngưỡng k và kéo dài đủ lâu thì luật đánh dấu sự kiện. Ý tưởng này em tham khảo từ ví dụ dms_system trong SDK của K230 và có hai thay đổi. Thứ nhất, khoảng cách được chia cho một độ dài (chiều rộng mặt hoặc khoảng cách mũi - tai) thay vì diện tích khuôn mặt như trong ví dụ gốc, vì độ dài tỉ lệ với căn bậc hai của diện tích. Thứ hai, trường hợp bàn tay ở gần mắt hơn tai và miệng được bỏ qua để không nhầm với dụi mắt. Riêng việc nhắn tin không đưa tay lên mặt nên được phát hiện qua việc cúi đầu kéo dài.

2.5 Ngân sách chú ý và máy trạng thái

Nếu bộ đếm thời gian nhìn lệch quay về 0 mỗi khi tài xế liếc lên đường, thì người vừa lái vừa nhắn tin và thỉnh thoảng nhìn lên sẽ không bao giờ bị cảnh báo. Dự án openpilot giải quyết vấn đề này bằng một đại lượng mức độ chú ý giảm dần khi tài xế mất tập trung và tăng dần khi tài xế chú ý trở lại. Em áp dụng ý tưởng tương tự: ngân sách A nằm trong khoảng từ 0 đến 1. Khi mất tập trung, A giảm với tốc độ w/T, trong đó T = 6 giây và w là trọng số theo nguyên nhân. Khi tài xế nhìn lại đường, A tăng với hệ số từ 1,25 đến 5 lần, tăng nhanh hơn khi A đang thấp. Mức cảnh báo

được xác định theo các ngưỡng 0,75; 0,58 và 0,33, còn A bằng 0 ứng với mức 4. Với w = 1, các mức lần lượt đạt sau 1,5; 2,5; 4,0 và 6,0 giây mất tập trung liên tục.

Mức cảnh báo hiển thị cho tài xế được đưa qua một máy trạng thái: mức tăng ngay khi điều kiện xảy ra, nhưng chỉ giảm từng bậc sau khi điều kiện đã hết một khoảng thời gian, để cảnh báo không bị nhấp nháy.

2.6 Hạ tầng dữ liệu của Traffic-SM

Traffic-SM nhận GPS của nhiều tài xế, khớp các điểm vào bản đồ đường thật bằng GraphHopper rồi tính mức tắc nghẽn. Dữ liệu đi qua Kafka theo các topic gps.raw, gps.traces và gps.matched. Spark Structured Streaming gom dữ liệu theo cửa sổ 10 giây (watermark 30 giây) cho từng đoạn đường, so sánh tốc độ trung vị với tốc độ chuẩn để gán mức FREE, SLOW hoặc CONGESTED. Kết quả được lưu trong ClickHouse và hiển thị bằng Grafana. Dữ liệu vào hệ thống qua cổng POST /v1/ingest, nhận một bản ghi hoặc một mảng tối đa 5.000 bản ghi. DriveFocus không sửa Traffic-SM: mỗi xe chỉ là thêm một nguồn GPS, với mã tài xế được thay bằng mã chuyến đi.

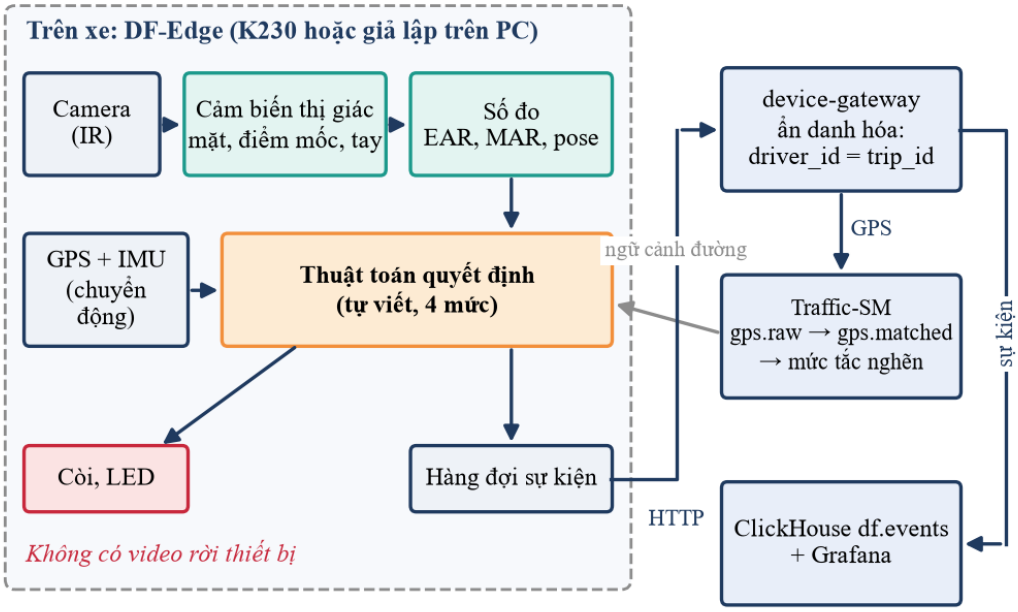
2.7 Chỉ số đánh giá

- Precision, recall và F1 cho từng luật. Precision là tỉ lệ ảnh bị đánh dấu đúng trong số ảnh bị đánh dấu, recall là tỉ lệ ảnh cần đánh dấu đã được đánh dấu, F1 là trung bình điều hoà của hai chỉ số này.
- AUC: xác suất để một ảnh thuộc lớp dương có điểm cao hơn một ảnh thuộc lớp âm, không phụ thuộc vào ngưỡng.
- Độ trễ cảnh báo: thời gian từ lúc bắt đầu nhắm mắt đến lúc đạt từng mức.
- Thời gian xử lý mỗi khung hình và số khung hình mỗi giây trên CPU.

CHƯƠNG 3: NỘI DUNG THỰC HIỆN VÀ KẾT QUẢ

3.1 Kiến trúc hệ thống

Hệ thống gồm ba khối (Hình 3.1). Trên xe, DF-Edge nhận ảnh từ camera và tốc độ từ GPS. Phần cảm biến tính các số đo hình học, phần quyết định đưa ra mức cảnh báo và lưu sự kiện vào hàng đợi. Sự kiện và GPS được gửi lên device-gateway. Gateway thay mã tài xế bằng mã chuyến đi, chuyển GPS sang Traffic-SM và lưu sự kiện vào ClickHouse. Video không được gửi ra khỏi thiết bị.



Hình 3.1: Kiến trúc hệ thống DriveFocus

Hình 3.2 là máy trạng thái của mức cảnh báo. Có 4 mức: nhắc nhẹ, cảnh báo, nguy hiểm và báo động. Với trường hợp nhắm mắt liên tục, các mức tương ứng với thời gian 0,5; 1; 2 và 3 giây.



Hình 3.2: Máy trạng thái hiển thị mức cảnh báo

3.2 Mã nguồn các bộ đo

Phần quyết định nằm trong gói drivefocus, khoảng 950 dòng Python, chỉ cần numpy và pyyaml. OpenCV và MediaPipe chỉ dùng ở phần cảm biến. Tất cả các ngưỡng được đặt trong một tệp cấu hình YAML để dễ thay đổi.

3.2.1 Tỉ số mở mắt

Hàm tính EAR cho một mắt nhận sáu điểm theo thứ tự: hai khoé mắt, cặp điểm thứ nhất và cặp điểm thứ hai (Hình 3.3). Nếu chiều rộng mắt bằng 0 thì hàm trả về None, tương ứng với trạng thái không xác định.

```
def eye_aspect_ratio(pts: Sequence[Sequence[float]]) -> Optional[float]:
    width = _dist(pts[0], pts[1])
    if width <= 1e-6:
        return None
    return (_dist(pts[2], pts[3]) + _dist(pts[4], pts[5])) / (2.0 * width)
```

Hình 3.3: Mã nguồn tính EAR từ sáu điểm theo toạ độ điểm ảnh

3.2.2 Thời gian nhắm mắt liên tục

Mức 4 được quyết định dựa vào thời gian nhắm mắt liên tục (Hình 3.4). Nếu có một vài khung hình mắt mở xen vào nhưng tổng cộng không quá 0,2 giây (thường do điểm mốc bị nhiễu) thì bộ đếm vẫn giữ nguyên. Khoảng không xác định ngắn cũng không làm thay đổi bộ đếm, nhưng nếu kéo dài quá 1 giây thì bộ đếm được đặt lại về 0.

```
def _update_run(self, t: float, closed: Optional[bool]) -> None:
    tol = self.c.closure_gap_tolerance_s
    if closed is None:
        if self._unknown_start is None:
            self._unknown_start = t
        if t - self._unknown_start > self.o.unknown_reset_s:
            self.reset_run()
        return
    self._unknown_start = None
    if closed:
        if self._run_start is None:
            self._run_start = t
        self._gap_start = None
        self.run = t - self._run_start
    else:
        if self._run_start is None:
            self.run = 0.0
            return
        if self._gap_start is None:
            self._gap_start = t
        if t - self._gap_start > tol:
            self.reset_run()
        else:
            self.run = t - self._run_start
```

Hình 3.4: Mã nguồn cập nhật thời gian nhắm mắt liên tục

3.2.3 Xác định xe đang chạy

Hệ thống chỉ cảnh báo khi xe đang chạy (Hình 3.5). Xe được coi là đang chạy khi tốc độ từ 10 km/h trở lên liên tục trong 3 giây, và được coi là dừng khi tốc độ dưới 3 km/h liên tục trong 5 giây. Hai ngưỡng khác nhau giúp trạng thái không bị bật tắt liên tục khi xe đi chậm trong lúc kẹt xe. Khi mất GPS, hệ thống giữ trạng thái cũ tối đa 30 giây rồi chuyển sang dùng cảm biến gia tốc (IMU). Nếu không có IMU thì trạng thái là chưa xác định, khi đó chỉ giữ lại cảnh báo mức 4 khi nhắm mắt.

```
def update(self, t: float, speed_kmh: Optional[float], vibration: Optional[float] = None) -
> str:
    c = self.c
    if speed_kmh is None:
        lost = float("inf") if self._last_fix_t is None else t - self._last_fix_t
        if lost <= c.gps_loss_hold_s:
            return self.state
        if vibration is not None:
            self.state = MOVING if vibration >= c.imu_vibration_thresh else STOPPED
        else:
            self.state = UNKNOWN
        self._enter_since = self._exit_since = None
        return self.state

    self._last_fix_t = t
    if self.state == UNKNOWN:
        self.state = STOPPED
    if self.state != MOVING:
        self._exit_since = None
        if speed_kmh >= c.enter_kmh:
            if self._enter_since is None:
                self._enter_since = t
            if t - self._enter_since >= c.enter_hold_s:
                self.state = MOVING
                self._enter_since = None
        else:
            self._enter_since = None
    else:
        self._enter_since = None
        if speed_kmh < c.exit_kmh:
            if self._exit_since is None:
                self._exit_since = t
            if t - self._exit_since >= c.exit_hold_s:
                self.state = STOPPED
                self._exit_since = None
        else:
            self._exit_since = None
    return self.state
```

Hình 3.5: Mã nguồn xác định trạng thái chuyển động của xe

3.2.4 Ngân sách chú ý

Hình 3.6 là phần cài đặt ngân sách chú ý. Mỗi nguyên nhân có một trọng số: nhìn lệch là 1,0, dùng điện thoại là 1,5 và uống nước hoặc ăn là 0,7.

```
def update(self, t: float, active: dict[str, bool], rate_div: float = 1.0) -> tuple[float,
int, Optional[str]]:
    dt = 0.0 if self._last is None else max(0.0, t - self._last)
    self._last = t
    causes = [k for k, v in active.items() if v]
    dominant: Optional[str] = None
    if causes:
        dominant = max(causes, key=lambda k: self.c.weights[k])
        w = self.c.weights[dominant]
        self.a -= w * dt / (self.c.t_empty_s * rate_div)
    else:
        fmin, fmax = self.c.recovery_factor
        f = fmin + (fmax - fmin) * (1.0 - self.a)
        self.a += f * dt / self.c.t_empty_s
    self.a = min(1.0, max(0.0, self.a))
    return self.a, self.level(), dominant
```

Hình 3.6: Mã nguồn ngân sách chú ý

3.3 Thực nghiệm và kết quả

3.3.1 Kiểm thử tự động

Em viết 50 bài kiểm thử cho phần thiết bị và 20 bài cho gateway, tất cả đều chạy đạt (Bảng 3.1). Ngoài các bài kiểm thử đơn vị, em dùng thêm một kịch bản kiểm thử ghi sẵn (golden trace) dài khoảng 60 giây gồm chớp mắt, nhắm mắt, nhìn lệch, dùng điện thoại và dừng xe. Kịch bản này được ghi ra tệp JSON Lines và phát lại, danh sách sự kiện đầu ra phải giống với tệp kết quả đã lưu. Khi chuyển chương trình sang chip K230, có thể dùng lại kịch bản này để kiểm tra phiên bản mới cho kết quả giống phiên bản trên máy tính.

Bảng 3.1: Các nhóm kiểm thử tự động

Nhóm	Số bài	Nội dung chính
Hình học	10	EAR, MAR, tư thế đầu và quy ước dấu của góc pitch
Bộ đo	16	Trạng thái chuyển động, nhắm mắt, PERCLOS, ngáp, tư thế, bàn tay, ngân sách chú ý
Quyết định	18	Kịch bản đầy đủ: mức 4 sau đúng 3 giây, chớp mắt, kính râm, xe dừng, mất mặt
Kịch bản ghi sẵn	2	Phát lại kịch bản và so sánh sự kiện
Cảm biến	4	Ảnh thật qua MediaPipe: EAR hợp lý, mắt nhắm vẫn quan sát được, vùng mắt tối thì không
Gateway	20	Định dạng dữ liệu, ẩn danh, loại trùng, hàng đợi khi mất mạng

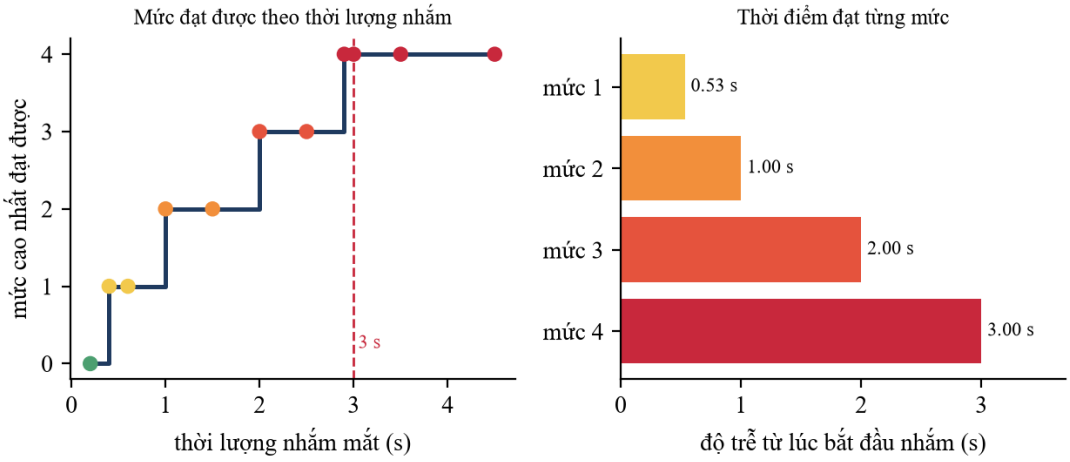
3.3.2 Thời gian nhắm mắt và mức cảnh báo

Do không có webcam và dữ liệu video nhắm mắt có nhãn, em dùng một ảnh chân dung thật và tạo thêm phiên bản mắt nhắm bằng cách phủ màu da lên vùng mắt. Hai ảnh này đi qua MediaPipe để lấy số đo thật: EAR khi mở mắt là 0,205, khi nhắm là 0,067 (bằng 0,33 lần). Từ hai bộ số đo này em tạo các chuỗi 15 khung/giây với thời gian nhắm mắt khác nhau rồi chạy phần quyết định. Thí nghiệm này chỉ dùng ảnh của một người nên chủ yếu kiểm tra logic ngưỡng và độ trễ (Bảng 3.2).

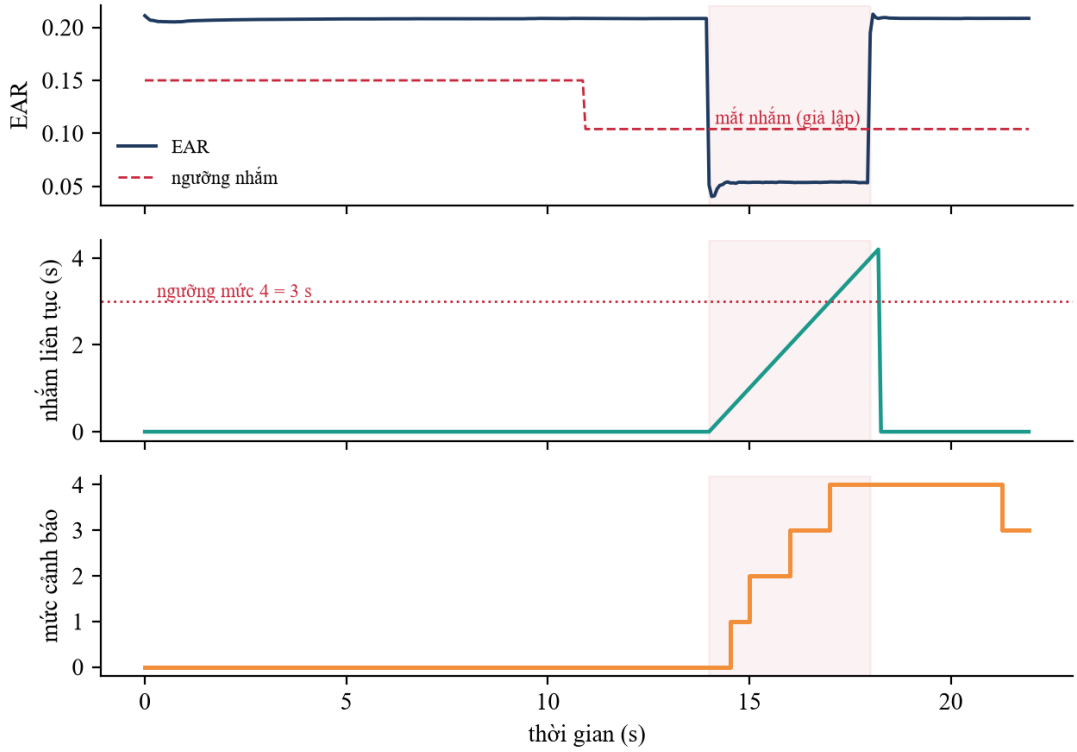
Bảng 3.2: Mức cảnh báo theo thời gian nhắm mắt (xe chạy 50 km/h)

Nhắm mắt (s)	Mức cao nhất	Đạt mức 1	Đạt mức 2	Đạt mức 3	Đạt mức 4
0,2	0	–	–	–	–
0,4	1	0,53 s	–	–	–
0,6	1	0,53 s	–	–	–
1,0	2	0,53 s	1,00 s	–	–
1,5	2	0,53 s	1,00 s	–	–
2,0	3	0,53 s	1,00 s	2,00 s	–
2,5	3	0,53 s	1,00 s	2,00 s	–
2,9	4	0,53 s	1,00 s	2,00 s	3,00 s
3,0	4	0,53 s	1,00 s	2,00 s	3,00 s
3,5	4	0,53 s	1,00 s	2,00 s	3,00 s
4,5	4	0,53 s	1,00 s	2,00 s	3,00 s

Kết quả đúng với thiết kế: mức 4 đạt tại thời điểm 3,00 giây, các mức 1, 2, 3 đạt tại khoảng 0,5; 1,0 và 2,0 giây (Hình 3.7). Do có khoảng dung sai 0,2 giây nên khi nhắm khoảng 2,9 giây hệ thống vẫn lên mức 4, và nhắm 0,4 giây đã lên mức 1. Hình 3.8 là các tín hiệu khi chạy với video thử gồm 14 giây mở mắt và 4 giây nhắm mắt: EAR giảm xuống dưới ngưỡng, thời gian nhắm tăng đều và mức cảnh báo tăng dần từng bậc.



Hình 3.7: Mức đạt được và độ trễ cảnh báo theo thời gian nhắm mắt



Hình 3.8: EAR, thời gian nhắm liên tục và mức cảnh báo trên video thử

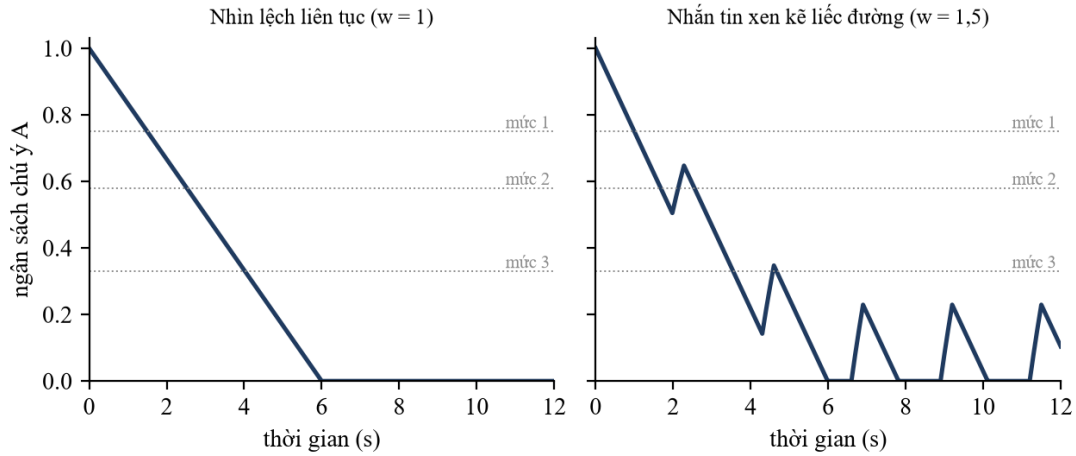
Để kiểm tra độ ổn định, em thêm nhiễu Gauss với độ lệch chuẩn 0,02 vào EAR và làm mất khuôn mặt ngẫu nhiên ở 3% số khung, mỗi trường hợp chạy 20 lần. Khi nhắm 0,3 giây, hệ thống không lần nào cảnh báo. Khi nhắm 1,0 giây, cả 20 lần đều lên mức 2. Khi nhắm 3,5 giây, 20/20 lần lên mức 4. Khi xe dừng và tài xế nhắm mắt 10 giây thì không có sự kiện nào.

Em cũng thử thêm hệ số theo tốc độ (mặc định tắt): ngưỡng thời gian được rút ngắn khi xe chạy nhanh, vì cùng một giây nhắm mắt thì xe đi được quãng đường xa hơn. Ở 50 km/h kết quả không đổi. Ở 100 km/h, nhắm mắt 1,5 giây chỉ lên mức 2 khi tắt hệ số, nhưng lên mức 4 sau 1,53 giây khi bật.

3.3.3 Ngân sách chú ý

Hình 3.9 so sánh hai trường hợp. Bên trái là tài xế nhìn lệch liên tục, ngân sách về 0 sau đúng 6 giây. Bên phải là tài xế nhắn tin, cứ khoảng 2 giây lại liếc lên đường 0,3 giây. Ngân sách chỉ hồi lại một phần nên vẫn giảm dần, trong khi nếu đặt lại bộ

đếm mỗi lần liếc đường thì trường hợp này sẽ không bao giờ bị cảnh báo. Bài kiểm thử tương ứng cũng cho thấy liếc gương dưới 1 giây không gây cảnh báo.



Hình 3.9: Ngân sách chú ý khi nhìn lệch liên tục và khi nhắn tin xen kẽ liếc đường

3.3.4 Đánh giá luật bàn tay gần đầu trên State Farm

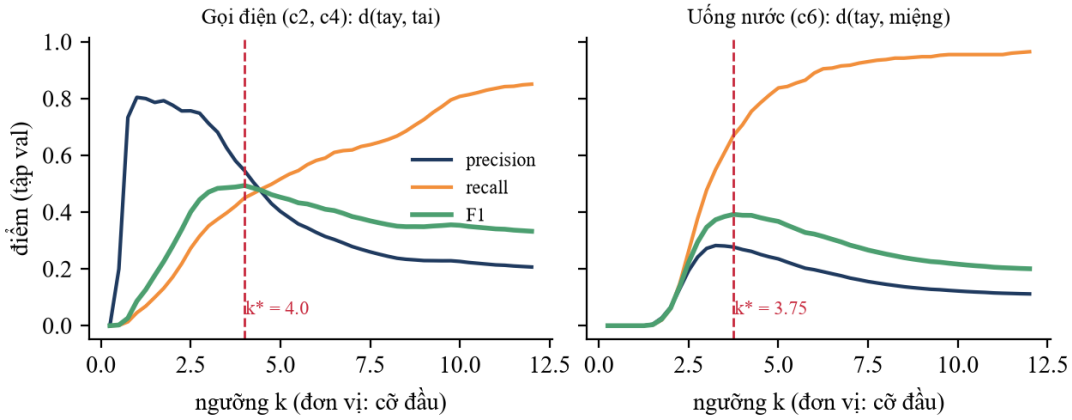
Vì ảnh State Farm chụp nghiêng, em dùng mô hình tư thế cơ thể để lấy vị trí tai, miệng và bàn tay. Tâm bàn tay là trung bình của cổ tay, ngón trỏ, ngón út và ngón cái. Khoảng cách từ tâm bàn tay đến tai (phía gần camera) và đến miệng được chia cho khoảng cách mũi - tai. Ngưỡng k được chọn trên tập val sao cho F1 lớn nhất, sau đó giữ nguyên để đánh giá trên tập test gồm các tài xế khác. Với luật tai, lớp dương là c2 và c4 (gọi điện); với luật miệng, lớp dương là c6 (uống nước); các lớp còn lại là lớp âm.

Bảng 3.3: Kết quả hai luật trên State Farm

Luật	k*	Val P / R / F1	Test P / R / F1	AUC val / test
Gọi điện (tay - tai)	4,00	0,55 / 0,45 / 0,49	0,55 / 0,25 / 0,34	0,69 / 0,75
Uống nước (tay - miệng)	3,75	0,28 / 0,67 / 0,39	0,16 / 0,23 / 0,19	0,78 / 0,65

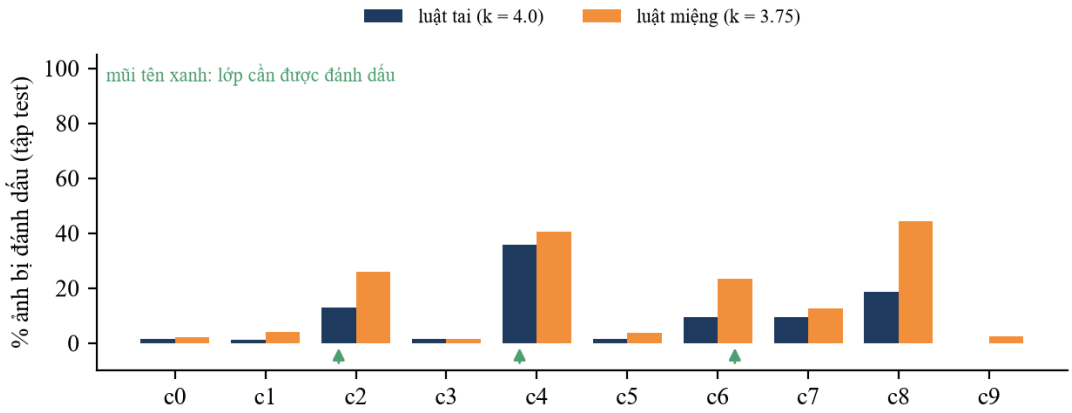
Kết quả chưa tốt: F1 trên tập test là 0,34 với luật gọi điện và 0,19 với luật uống nước, thấp hơn nhiều so với tập val vì tài xế ở tập test khác tài xế ở tập val. Hình 3.10 cho thấy ngưỡng tốt nhất nằm ở khoảng giữa của dải ngưỡng đã thử. Em thấy có hai nguyên nhân chính. Khi gọi điện bằng tay trái (c4), tay nằm ở phía tai xa camera nên

bị đầu che. Ngoài ra, khoảng cách mũi - tai trong ảnh chỉ khoảng 22 đến 41 điểm ảnh (trung vị theo lớp), quá nhỏ nên phép chuẩn hoá bị nhiễu.

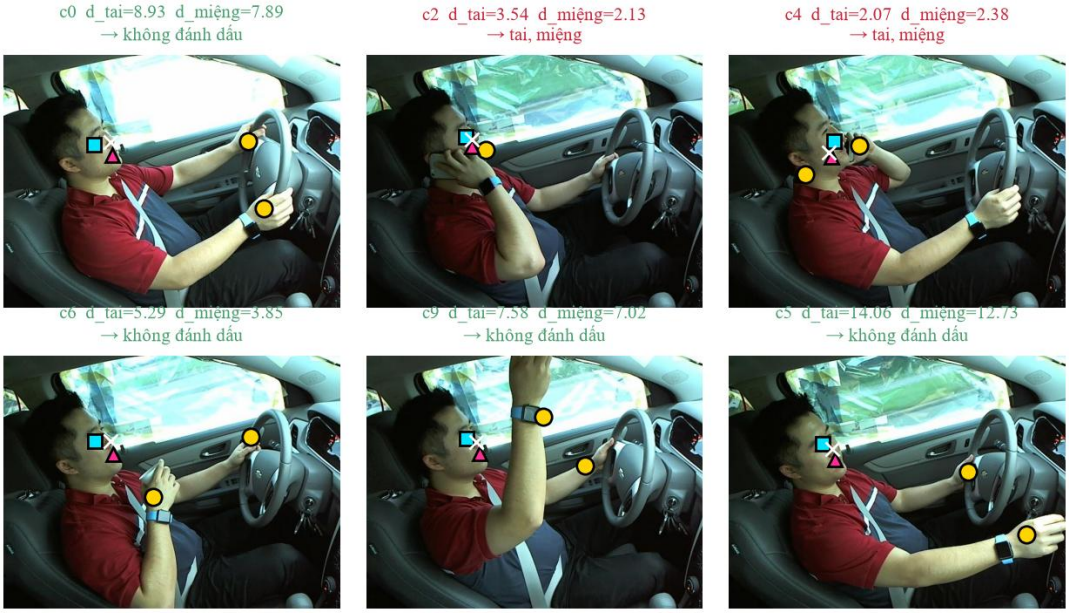


Hình 3.10: Precision, recall và F1 theo ngưỡng k trên tập val

Nếu xem luật như một bộ phát hiện chung cho việc đưa tay lên đầu thì kết quả khả quan hơn (Hình 3.11). Lớp c0 (lái xe an toàn) chỉ bị đánh dấu 2,1%, gần bằng các lớp c3 và c9 vốn không đưa tay lên đầu. Trong khi đó các lớp có đưa tay lên đầu bị đánh dấu nhiều hơn hẳn: c4 40,7%, c8 44,8%, c2 26,0% và c6 23,2%. Tuy vậy luật không phân biệt được gọi điện, uống nước và chải tóc vì đều là đưa tay lên đầu, và gần như không phát hiện được nhắn tin (c1, c3 dưới 5%) vì khi nhắn tin tay không đưa lên mặt.



Hình 3.11: Tỉ lệ ảnh bị đánh dấu theo từng lớp trên tập test

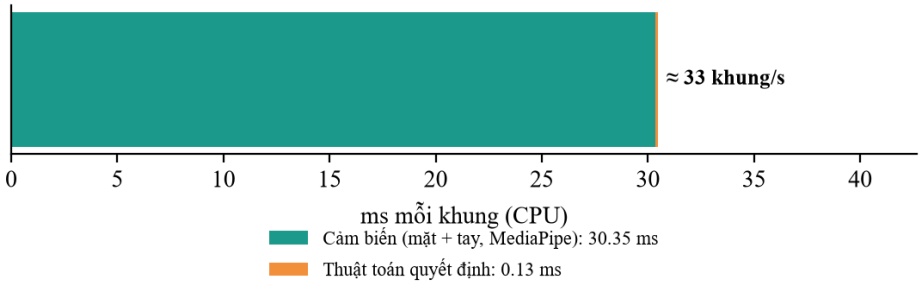


Hình 3.12: Một số kết quả: tai (vuông xanh), miệng (tam giác hồng), bàn tay (tròn vàng) và kết luận của luật

Trong bài tập lớn trước, mô hình EfficientNet-Lite0 với cùng cách chia tài xế đạt macro-F1 khoảng 0,82 cho bài toán sáu nhóm hành vi. Hai kết quả không so sánh trực tiếp được vì bài toán khác nhau, nhưng có thể thấy luật hình học đơn giản kém hơn nhiều so với một mô hình được huấn luyện trên chính bộ dữ liệu này. Bù lại, luật không cần dữ liệu huấn luyện, dễ giải thích và có thể dùng cho camera đặt ở góc khác chỉ bằng cách chỉnh lại ngưỡng, còn mô hình huấn luyện trên State Farm sẽ không còn chính xác khi đổi góc camera.

3.3.5 Hiệu năng

Em đo thời gian xử lý trên CPU 12th Gen Intel(R) Core(TM) i5-12400F, Python 3.11.9, MediaPipe 1.2.2, với video có khung hình 820×1024, tổng cộng 320 khung (bỏ 10 khung đầu). Phần cảm biến (khuôn mặt và bàn tay) mất trung bình 30,3 ms mỗi khung (p95 là 32,5 ms), phần quyết định chỉ mất 0,13 ms. Cả hệ thống đạt khoảng 33 khung/giây (Hình 3.13). Như vậy phần quyết định không ảnh hưởng đến tốc độ chung. Hiệu năng trên chip K230 em chưa đo được.



Hình 3.13: Thời gian xử lý mỗi khung trên CPU

Bảng 3.4: Hiệu năng đo trên CPU

Hạng mục	Giá trị
Cảm biến khuôn mặt và bàn tay, trung bình mỗi khung	30,3 ms
Cảm biến, p95	32,5 ms
Phần quyết định, mỗi khung	0,13 ms
Toàn hệ thống	khoảng 33 khung/giây
Gateway chuyển đổi GPS	khoảng 155.000 điểm/giây

3.4 Phần máy chủ và kết nối với Traffic-SM

Device-gateway nhận GPS từ thiết bị ở dạng rút gọn (thời điểm, vĩ độ, kinh độ, tốc độ) và chuyển sang định dạng mà cổng nhận dữ liệu của Traffic-SM yêu cầu. Cổng này kiểm tra khá chặt: phải có đủ mười một trường số, trường time theo giờ UTC+7, phút trong t_timestamp phải trùng với phút trong time, và vehicle_type chỉ nhận car hoặc bike. Để kiểm tra phần chuyển đổi, em chạy lại trên 200 dòng thật của fake_gps.csv: với cùng một thời điểm, gateway tạo ra đúng giá trị time và t_timestamp giống như Traffic-SM (Hình 3.14).

```
def to_ingest_payload(point: dict[str, Any], trip_id: str, vehicle_type: str,
                      prev: Optional[dict[str, Any]]) -> dict[str, Any]:
    ts = _finite(point.get("ts"), "ts")
    lat = _finite(point.get("lat"), "lat")
    lng = _finite(point.get("lng"), "lng")
    speed = max(0.0, _finite(point.get("speed_mps"), "speed_mps"))
    if ts <= 0:
        raise ValidationError("ts phải dương")
    if not -90 <= lat <= 90 or not -180 <= lng <= 180:
        raise ValidationError("lat/lng ngoài phạm vi")


    local = datetime.fromtimestamp(ts, tz=VN_TZ)
    bearing = point.get("bearing")
    if bearing is not None:
        bearing = _finite(bearing, "bearing")
    if bearing is None:
        bearing = -1.0
        if prev and speed >= 0.5 and haversine_m(prev["lat"], prev["lng"], lat, lng) >=
1.0:
            bearing = round(bearing_deg(prev["lat"], prev["lng"], lat, lng), 2)
    dt = point.get("time_delta")
    if dt is not None:
        dt = _finite(dt, "time_delta")
    else:
        dt = max(0.0, ts - prev["ts"]) if prev else 1.0
    return {
        "driver_id": trip_id,
        "t_timestamp": f"{local.minute:02d}:{local.second:02d}.{local.microsecond //
100000}",
        "lat": lat, "lng": lng,
        "bearing": bearing,
        "bearing_acc": _finite(point.get("bearing_acc", 10.0), "bearing_acc"),
        "horizontal_acc": _finite(point.get("horizontal_acc", 5.0), "horizontal_acc"),
        "speed": round(speed, 3),
        "speed_acc": _finite(point.get("speed_acc", 0.5), "speed_acc"),
        "time_delta": dt,
        "vertical_acc": _finite(point.get("vertical_acc", 5.0), "vertical_acc"),
        "status": "IN TRIP",
        "vehicle_type": vehicle_type,
        "time": f"{local.month}/{local.day}/{local.year} {local.hour}:{local.minute:02d}",
        "timestamp": ts,
        "altitude": _finite(point.get("altitude", 0.0), "altitude"),
    }
```

Hình 3.14: Mã nguồn chuyển một điểm GPS của thiết bị sang bản ghi của Traffic-SM

Trước khi chuyển dữ liệu đi, gateway bỏ mã thiết bị. Traffic-SM và bảng sự kiện chỉ thấy mã chuyến đi, còn bảng ghép chuyến đi với thiết bị được lưu riêng trong SQLite. Sự kiện được loại trùng theo mã sự kiện nên khi thiết bị gửi lại thì không bị ghi hai lần. Khi mất mạng, thiết bị ghi dữ liệu vào tệp outbox và gửi lại ở lần chạy sau.

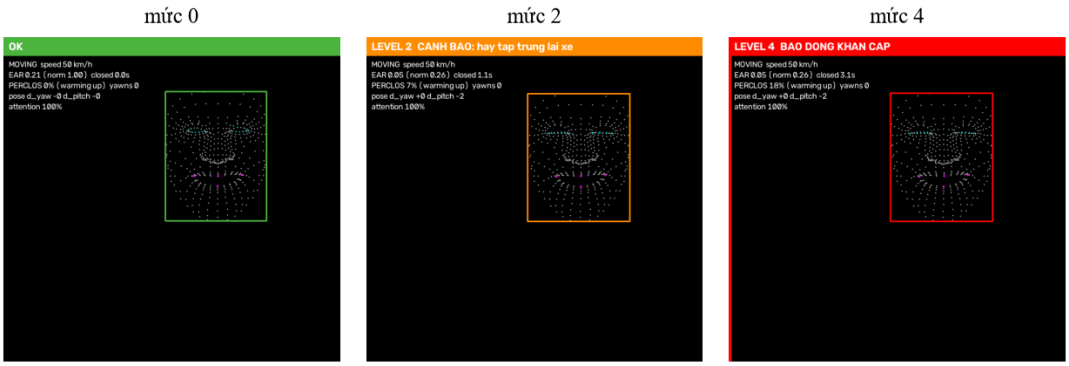
Bảng 3.5: Kết quả chạy thử toàn bộ luồng: video, thiết bị, gateway, máy chủ nhận dữ liệu giả lập

Hạng mục	Kết quả
Điểm GPS gửi qua gateway	49 điểm của 3 chuyến, nhận 49, từ chối 0
Sự kiện được lưu	6 sự kiện, không có mã thiết bị, không trùng
Gửi lại từ outbox khi gateway chạy lại	7 bản ghi gửi thành công, outbox còn 0
Mã tài xế phía Traffic-SM nhận được	là mã chuyến đi (3 mã khác nhau)
Chạy với Traffic-SM thật	chưa chạy (máy chưa chạy Docker, thiếu dữ liệu bản đồ OSM)

Phía nhận dữ liệu trong lần chạy thử là một máy chủ giả lập do em viết, kiểm tra dữ liệu theo đúng các quy tắc trong hàm gps.Canonicalize của Traffic-SM (viết bằng Go). Các phần còn lại của Traffic-SM như map matching, Spark, ClickHouse và Grafana chưa được chạy lại trong đồ án này.

3.5 Sản phẩm

Chương trình demo chạy trên máy tính với webcam hoặc tệp video (Hình 3.15). Cửa sổ hiển thị mức cảnh báo bằng thanh màu ở phía trên, các số đo EAR, PERCLOS, tư thế đầu, ngân sách chú ý, trạng thái chuyển động của xe và các cờ báo chất lượng. Ảnh trong báo cáo được vẽ ở chế độ riêng tư, chỉ có các điểm mốc trên nền đen. Chương trình phát âm thanh bằng winsound trên Windows, mỗi mức có một kiểu âm báo riêng. Các phím tắt: w và s để tăng giảm tốc độ giả lập, phím cách để cho xe dừng hoặc chạy, c để hiệu chỉnh nhanh, m để tắt âm thanh và q để thoát.



Hình 3.15: Giao diện demo ở chế độ riêng tư tại mức 0, mức 2 và mức 4

Cách chạy: lệnh python -m drivefocus.demo --quick để chạy với webcam; thêm --source clip.mp4 --no-window để chạy với tệp video; thêm --route-csv và --gateway để gửi dữ liệu qua gateway; lệnh python -m drivefocus.replay trace.jsonl để phát lại kịch bản đã ghi.

KẾT LUẬN

Trong đồ án 3, em đã xây dựng hệ thống giám sát tài xế DriveFocus và đạt được một số kết quả sau:

- Xây dựng phần quyết định cảnh báo bằng thuật toán, không dùng mô hình phân loại hành vi: EAR chuẩn hoá theo từng người, PERCLOS tính theo thời gian, tư thế đầu từ ma trận biến đổi, luật bàn tay gần đầu, ngân sách chú ý, xác định xe đang chạy và máy trạng thái bốn mức.
- Khi nhắm mắt liên tục 3 giây, hệ thống lên mức 4 đúng tại thời điểm 3,00 giây; không cảnh báo khi chỉ chớp mắt, vẫn ổn định khi có nhiễu và mất khuôn mặt ngẫu nhiên, và không cảnh báo khi xe dừng.
- Viết 70 bài kiểm thử tự động (đều đạt) và một kịch bản ghi sẵn để kiểm tra khi chuyển sang thiết bị khác.
- Phần quyết định chỉ mất khoảng 0,13 ms mỗi khung; cả hệ thống chạy khoảng 33 khung/giây trên CPU máy tính.
- Xây dựng phần máy chủ tối thiểu: ẩn danh dữ liệu, chuyển GPS sang định dạng của Traffic-SM (đã kiểm tra với 200 dòng dữ liệu thật), loại trùng sự kiện và gửi lại khi mất mạng.
- Đánh giá trên State Farm: mô hình khuôn mặt chỉ tìm được mặt ở khoảng 28% số ảnh do góc chụp nghiêng; luật bàn tay gần đầu đạt F1 0,34 với gọi điện và 0,19 với uống nước.

Đồ án vẫn còn một số hạn chế:

- Chưa thử với webcam và người lái thật. Thí nghiệm nhắm mắt mới dùng ảnh của một người, vùng mắt nhắm được tạo bằng cách phủ màu da. Các ngưỡng hiện tại mới là giá trị ban đầu.
- Chưa có dữ liệu mặt nhìn thẳng có nhãn (NTHU-DDD, YawDD, DMD) nên chưa đánh giá được EAR, PERCLOS và ngáp trên nhiều người, chưa đo được số lần báo nhầm khi lái xe bình thường.
- Luật bàn tay gần đầu còn yếu: chưa phân biệt được các hành vi đưa tay lên đầu và không phát hiện được nhắn tin.

- Chưa chạy trên chip K230 nên hiệu năng và độ chính xác trên chip vẫn chỉ là ước lượng.
- Chưa chạy với Traffic-SM thật, mới thử với máy chủ giả lập, và chưa có phần bảo mật (mã hoá, xác thực thiết bị).

Trong thời gian tới, em dự định tải và đánh giá trên các bộ NTHU-DDD, YawDD và DMD để chỉnh lại ngưỡng, quay thêm video ở đúng vị trí đặt camera trên xe, chuyển phần cảm biến sang các mô hình trong SDK của K230, kết nối với Traffic-SM thật để làm bản đồ rủi ro theo đoạn đường, và bổ sung phần bảo mật (mTLS, phân quyền, cập nhật có chữ ký).

TÀI LIỆU THAM KHẢO

1) T. Soukupová, J. Čech, “Real-Time Eye Blink Detection using Facial Landmarks”, Computer Vision Winter Workshop, 2016.
2) D. F. Dinges, R. Grace, “PERCLOS: A Valid Psychophysiological Measure of Alertness as Assessed by Psychomotor Vigilance”, FHWA-MCRT-98-006, 1998.
3) C. Lugaresi và cộng sự, “MediaPipe: A Framework for Building Perception Pipelines”, arXiv:1906.08172, 2019.
4) Y. Kartynnik và cộng sự, “Real-time Facial Surface Geometry from Monocular Video on Mobile GPUs”, CVPR Workshops, 2019.
5) F. Zhang và cộng sự, “MediaPipe Hands: On-device Real-time Hand Tracking”, arXiv:2006.10214, 2020.
6) V. Bazarevsky và cộng sự, “BlazePose: On-device Real-time Body Pose tracking”, arXiv:2006.10204, 2020.
7) V. Lepetit, F. Moreno-Noguer, P. Fua, “EPnP: An Accurate O(n) Solution to the PnP Problem”, International Journal of Computer Vision, 2009.
8) G. Bradski, “The OpenCV Library”, Dr. Dobb’s Journal of Software Tools, 2000.
9) J. Kreps, N. Narkhede, J. Rao, “Kafka: a Distributed Messaging System for Log Processing”, NetDB, 2011.
10) M. Zaharia và cộng sự, “Apache Spark: A Unified Engine for Big Data Processing”, Communications of the ACM, 2016.
11) State Farm Distracted Driver Detection, Kaggle, 2016: https://www.kaggle.com/c/state-farm-distracted-driver-detection
12) Dự án openpilot (comma.ai), mô-đun giám sát tài xế: https://github.com/commaai/openpilot
13) Dự án driver-state-detection (e-candeloro): https://github.com/e-candeloro/driver-state-detection
14) Kendryte K230 SDK và nncase: https://github.com/kendryte/k230_sdk, https://github.com/kendryte/nncase
15) Tài liệu MediaPipe: https://ai.google.dev/edge/mediapipe
16) Báo cáo bài tập lớn môn Thị giác máy tính: Nhận diện hành vi tài xế mất tập trung (dds-edge), 2026.
17) Traffic-SM: hệ thống phân tích tắc nghẽn giao thông từ GPS (Kafka, Spark, ClickHouse, GraphHopper), 2026.